P89. 1

分别试用（1）Jacobi 迭代法；（2）Gauss-Seidel 迭代法；（3）共轭梯度法解线性方程组

$$\begin{pmatrix} 10 & 1 & 2 & 3 & 4 \\ 1 & 9 & -1 & 2 & -3 \\ 2 & -1 & 7 & 3 & -5 \\ 3 & 2 & 3 & 12 & -1 \\ 4 & -3 & -5 & -1 & 15 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} = \begin{pmatrix} 12 \\ -27 \\ 14 \\ -17 \\ 12 \end{pmatrix}.$$

迭代初始向量取 $\boldsymbol{x}^{(0)} = (0, 0, 0, 0, 0)^{\mathrm{T}}$。

解：实验内容、步骤及结果

先编写 Jacobi 迭代函数 `chap-3/jacobi.m`：

```
function [x, iter, last_error] = jacobi(A, b, max_iter, x0, tol)
    % Jacobi Iteration Method
    % Inputs:
    % A - Coefficient matrix
    % b - Right-hand side vector
    % max_iter - Maximum number of iterations
    % x0 - Initial guess (optional)
    % tol - Tolerance for convergence (optional)
    % Outputs:
    % x - Solution vector
    % iter - Number of iterations performed
    % last_error - Error of the last iteration

    % Set default initial guess
    if nargin < 4 || isempty(x0)
        x0 = zeros(size(b));
    end

    % Set default tolerance
    if nargin < 5 || isempty(tol)
        tol = 0;
    end

    % Initial guess
    x = x0;
    n = length(b);
    iter = 0;
    last_error = inf;

    % Iterate
```

```
    for k = 1:max_iter
        x_new = zeros(size(x));
        for i = 1:n
            sum = 0;
            for j = 1:n
                if j ~= i
                    sum = sum + A(i, j) * x(j);
                end
            end
            x_new(i) = (b(i) - sum) / A(i, i);
        end

        % Check for convergence
        last_error = norm(x_new - x, inf);
        if tol > 0
            if last_error < tol
                x = x_new;
                iter = k;
                return;
            end
        end

        x = x_new;
        iter = k;
    end

    warning('Jacobi method did not converge within the maximum number of
        iterations');
end
```

再编写 Gauss-Seidel 迭代函数 chap-3/gauss_seidel.m:

```
function [x, iter, last_error] = gauss_seidel(A, b, max_iter, x0, tol)
    % Gauss-Seidel Iteration Method
    % Inputs:
    % A - Coefficient matrix
    % b - Right-hand side vector
    % max_iter - Maximum number of iterations
    % x0 - Initial guess (optional)
    % tol - Tolerance for convergence (optional)
    % Outputs:
    % x - Solution vector
    % iter - Number of iterations performed
    % last_error - Error of the last iteration
```

```
% Set default initial guess
if nargin < 4 || isempty(x0)
    x0 = zeros(size(b));
end

% Set default tolerance
if nargin < 5 || isempty(tol)
    tol = 0;
end

% Initial guess
x = x0;
n = length(b);
iter = 0;
last_error = inf;

% Iterate
for k = 1:max_iter
    x_old = x;
    for i = 1:n
        sum = 0;
        for j = 1:n
            if j ~= i
                sum = sum + A(i, j) * x(j);
            end
        end
        x(i) = (b(i) - sum) / A(i, i);
    end

    % Check for convergence
    last_error = norm(x - x_old, inf);
    if tol > 0
        if last_error < tol
            iter = k;
            return;
        end
    end

    iter = k;
end

warning('Gauss-Seidel method did not converge within the maximum number
```

```
        of iterations');
end
```

共轭梯度法函数 chap-3/cg.m:

```
function [x, iter, last_error] = cg(A, b, max_iter, x0, tol)
    % Conjugate Gradient Method
    % Inputs:
    % A - Coefficient matrix (must be symmetric positive definite)
    % b - Right-hand side vector
    % max_iter - Maximum number of iterations
    % x0 - Initial guess (optional)
    % tol - Tolerance for convergence (optional)
    % Outputs:
    % x - Solution vector
    % iter - Number of iterations performed
    % last_error - Error of the last iteration

    % Set default initial guess
    if nargin < 4 || isempty(x0)
        x0 = zeros(size(b));
    end

    % Set default tolerance
    if nargin < 5 || isempty(tol)
        tol = 0;
    end

    % Initial guess
    x = x0;
    r = b - A * x;
    p = r;
    rsold = r' * r;
    iter = 0;
    last_error = inf;

    for k = 1:max_iter
        Ap = A * p;
        alpha = rsold / (p' * Ap);
        x = x + alpha * p;
        r = r - alpha * Ap; % r = b - A * x
        rsnew = r' * r;
        last_error = sqrt(rsnew);
```

```
        % Check for convergence
        if tol > 0
            if last_error < tol
                iter = k;
                return;
            end
        end

        p = r + (rsnew / rsold) * p;
        rsold = rsnew;
        iter = k;
    end

    warning('Conjugate Gradient method did not converge within the maximum
        number of iterations');
end
```

主程序 chap-3/P89_Q1.m:

```
% Define the coefficient matrix A and the right-hand side vector b
A = [10, 1, 2, 3, 4; 1, 9, -1, 2, -3; 2, -1, 7, 3, -5; 3, 2, 3, 12, -1; 4,
    -3, -5, -1, 15];
b = [12; -27; 14; -17; 12];

max_iter = inf;
x0 = zeros(size(b));
epsilon = 1e-16;

% Call the Jacobi function
[x, iter, last_error] = jacobi(A, b, max_iter, x0, epsilon);
disp('--- Jacobi Iteration Method Outp. Start ---');
disp('Solution vector x:');
disp(x);
disp('Number of iterations:');
disp(iter);
disp('Error of the last iteration:');
disp(last_error);
disp('--- Jacobi Iteration Method Outp. End ---');

% Call the Gauss-Seidel function
[x, iter, last_error] = gauss_seidel(A, b, max_iter, x0, epsilon);
disp('--- Gauss-Seidel Iteration Method Outp. Start ---');
disp('Solution vector x:');
disp(x);
```

```
disp('Number of iterations:');
disp(iter);
disp('Error of the last iteration:');
disp(last_error);
disp('--- Gauss-Seidel Iteration Method Outp. End ---');

% Call the CG function
[x, iter, last_error] = cg(A, b, max_iter, x0, epsilon);
disp('--- CG Method Outp. Start ---');
disp('Solution vector x:');
disp(x);
disp('Number of iterations:');
disp(iter);
disp('Error of the last iteration:');
disp(last_error);
disp('--- CG Method Outp. End ---');
```

实验结果分析

```
>> P89_Q1
Warning: Too many FOR loop iterations. Stopping after 9223372036854775806
    iterations.
> In jacobi (line 45)
In P89_Q1 (line 10)

--- Jacobi Iteration Method Outp. Start ---
Solution vector x:
     1
    -2
     3
    -2
     1

Number of iterations:
   190

Error of the last iteration:
     0

--- Jacobi Iteration Method Outp. End ---
Warning: Too many FOR loop iterations. Stopping after 9223372036854775806
    iterations.
> In gauss_seidel (line 45)
In P89_Q1 (line 21)
```

```
--- Gauss-Seidel Iteration Method Outp. Start ---
Solution vector x:
     1
    -2
     3
    -2
     1

Number of iterations:
    99

Error of the last iteration:
     0

--- Gauss-Seidel Iteration Method Outp. End ---
Warning: Too many FOR loop iterations. Stopping after 9223372036854775806
   iterations.
> In cg (line 46)
In P89_Q1 (line 32)

--- CG Method Outp. Start ---
Solution vector x:
    1.0000
   -2.0000
    3.0000
   -2.0000
    1.0000

Number of iterations:
     9

Error of the last iteration:
   2.4446e-17

--- CG Method Outp. End ---
```

三种方法都求得相同解

$$x = (1, -2, 3, -2, 1)^{\mathrm{T}}.$$

从迭代步数可以看出：共轭梯度法仅用 9 步即达到极高精度，收敛速度最快；Gauss-Seidel 迭代需要 99 步；Jacobi 迭代最慢，需要 190 步。

共轭梯度法最初是为大型稀疏对称正定方程组设计的。它与二次型

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^{\mathrm{T}} A\mathbf{x} - \mathbf{b}^{\mathrm{T}}\mathbf{x}$$

的最优化问题密切相关，因为最优点满足

$$\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b} = 0,$$

这正对应线性方程组 $A\mathbf{x} = \mathbf{b}$。因此，共轭梯度法可以看成是在对称正定二次型上沿 A-共轭方向逐步逼近极小点的过程。

P89. 2

若仅保存系数矩阵 $\boldsymbol{A} = \langle a_{ij}\rangle_{n\times n}$ 的非零元，用共轭梯度法求解 $n = 10^5$ 阶的方程组 $\mathbf{Ax} = \mathbf{b}$，其中

$$a_{ij} = \begin{cases} 3, & i = j, \\ -1, & |i-j| = 1, \\ \dfrac{1}{2}, & i + j = n+1,\ i \neq \dfrac{n}{2},\ \dfrac{n}{2}+1, \\ 0, & \text{其他}, \end{cases} \qquad (i, j = 1, 2, \ldots, n)$$

且

$$b = (2.5, 1.5, \ldots, 1.5, 1.0, 1.0, 1.5, \ldots, 1.5, 2.5)^{\mathrm{T}}.$$

解： 实验内容、步骤及结果

程序 `chap-3/P89_Q2.m`:

```
n = 10 ^ 5;
disp(['n: ', num2str(n)]);

% Initialize matrix A with zeros
if n <= 10
    A = zeros(n, n);
else
    A = sparse(n, n);
end

% Fill the diagonal with 3
for i = 1:n
    A(i, i) = 3;
end

% Fill the sub-diagonal and super-diagonal with -1
for i = 1:n - 1
    A(i, i + 1) = -1;
```

```
    A(i + 1, i) = -1;
end

% Fill the anti-diagonal with 1/2, except for the middle elements
for i = 1:n
    j = n + 1 - i;

    if i ~= n / 2 && i ~= n / 2 + 1
        A(i, j) = 0.5;
    end

end

if n <= 10
    disp('A');
    disp(A);
end

% Initialize vector b
b = ones(n, 1) * 1.5;
b(1) = 2.5;
b(n) = 2.5;
b(floor(n / 2)) = 1.0;
b(floor(n / 2) + 1) = 1.0;

if n <= 10
    disp('b');
    disp(b);
end

max_iter = inf;
x0 = zeros(size(b));
epsilon = 1e-16;

% Call the CG function
[x, iter, last_error] = cg(A, b, max_iter, x0, epsilon);
disp('--- CG Method Outp. Start ---');
if n <= 10
    disp('Solution vector x:');
    disp(x);
else
    disp('Solution vector x: (only the first 10 elements)');
    disp(x(1:10));
```

```
end
disp('Number of iterations:');
disp(iter);
disp('Error of the last iteration:');
disp(last_error);
disp('--- CG Method Outp. End ---');
```

为了便于观察结构与结果，先给出 $n = 10$ 时的输出：

```
>> P89_Q2
n: 10
A
    3.0000   -1.0000         0         0         0         0         0
                 0         0    0.5000
   -1.0000    3.0000   -1.0000         0         0         0         0
                 0    0.5000         0
         0   -1.0000    3.0000   -1.0000         0         0         0
              0.5000         0         0
         0         0   -1.0000    3.0000   -1.0000         0    0.5000
                        0         0         0
         0         0         0   -1.0000    3.0000   -1.0000         0
                        0         0         0
         0         0         0         0   -1.0000    3.0000   -1.0000
                        0         0         0
         0         0         0    0.5000         0   -1.0000    3.0000
              -1.0000         0         0
         0         0    0.5000         0         0         0   -1.0000
              3.0000   -1.0000         0
         0    0.5000         0         0         0         0         0
              -1.0000    3.0000   -1.0000
    0.5000         0         0         0         0         0         0
                   0   -1.0000    3.0000

b
    2.5000
    1.5000
    1.5000
    1.5000
    1.0000
    1.0000
    1.5000
    1.5000
    1.5000
    2.5000
```

```
--- CG Method Outp. Start ---
Solution vector x:
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000

Number of iterations:
     5

Error of the last iteration:
   2.3716e-17

--- CG Method Outp. End ---
```

当 $n = 10^5$ 时, 程序输出为:

```
>> P89_Q2
n: 100000
Warning: Too many FOR loop iterations. Stopping after 9223372036854775806
    iterations.
> In cg (line 46)
In P89_Q2 (line 54)

--- CG Method Outp. Start ---
Solution vector x: (only the first 10 elements)
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
    1.0000
```

```
Number of iterations:
    35

Error of the last iteration:
   4.3128e-17

--- CG Method Outp. End ---
```

实验结果分析

从 $n = 10$ 和 $n = 10^5$ 两组结果都可以看出，解向量都趋于全 1 向量。即使矩阵阶数极大，只要利用稀疏存储保留非零元，共轭梯度法仍能在较少步数内完成求解；本实验中当 $n = 10^5$ 时，仅用 35 步就达到很高精度，说明共轭梯度法非常适合求解大型稀疏对称正定线性方程组。

补充代码：P87. Q2, Q7

以下补充第三章作业中涉及的 SOR 迭代实现，以及 P87.2、P87.7 对应的 MATLAB 程序。

解： chap-3/sor.m:

```
function [x, iter, last_error] = sor(A, b, omega, max_iter, x0, tol)
    % Successive Over-Relaxation (SOR) Iteration Method
    % Inputs:
    % A - Coefficient matrix
    % b - Right-hand side vector
    % omega - Relaxation factor
    % max_iter - Maximum number of iterations
    % x0 - Initial guess (optional)
    % tol - Tolerance for convergence (optional)
    % Outputs:
    % x - Solution vector
    % iter - Number of iterations performed
    % last_error - Error of the last iteration

    % Set default initial guess
    if nargin < 5 || isempty(x0)
        x0 = zeros(size(b));
    end

    % Set default tolerance
    if nargin < 6 || isempty(tol)
        tol = 0;
    end

    % Initial guess
```

```
    x = x0;
    n = length(b);
    iter = 0;
    last_error = inf;

    % Iterate
    for k = 1:max_iter
        x_old = x;
        for i = 1:n
            sum1 = 0;
            sum2 = 0;
            for j = 1:i-1
                sum1 = sum1 + A(i, j) * x(j);
            end
            for j = i+1:n
                sum2 = sum2 + A(i, j) * x_old(j);
            end
            x(i) = (1 - omega) * x_old(i) + omega * (b(i) - sum1 - sum2) /
                A(i, i);
        end

        % Check for convergence
        last_error = norm(x - x_old, inf);
        if tol > 0
            if last_error < tol
                iter = k;
                return;
            end
        end

        iter = k;
    end

    warning('SOR method did not converge within the maximum number of
        iterations');
end
```

chap-3/P87_Q2.m:

```
% Define the coefficient matrix A and the right-hand side vector b
A = [2, -1, 0, 0; -1, 2, -1, 0; 0, -1, 2, -1; 0, 0, -1, 2];
b = [1; 0; 1; 0];

% Call the Jacobi function
```

```
for max_iter = [1, 2, 3, inf]
    [x, iter, last_error] = jacobi(A, b, max_iter, ones(size(b)), 1e-3);
    disp('--- Jacobi Iteration Method Outp. Start ---');
    disp('Solution vector x:');
    disp(x);
    disp('Number of iterations:');
    disp(iter);
    disp('Error of the last iteration:');
    disp(last_error);
    disp('--- Jacobi Iteration Method Outp. End ---');
end

% Call the Gauss-Seidel function
for max_iter = [1, 2, 3, inf]
    [x, iter, last_error] = gauss_seidel(A, b, max_iter, ones(size(b)), 1e
        -3);
    disp('--- Gauss-Seidel Iteration Method Outp. Start ---');
    disp('Solution vector x:');
    disp(x);
    disp('Number of iterations:');
    disp(iter);
    disp('Error of the last iteration:');
    disp(last_error);
    disp('--- Gauss-Seidel Iteration Method Outp. End ---');
end

% Call the SOR function
for max_iter = [1, 2, 3, inf]
    [x, iter, last_error] = sor(A, b, 1.46, max_iter, ones(size(b)), 1e-3);
    disp('--- SOR Iteration Method Outp. Start ---');
    disp('Solution vector x:');
    disp(x);
    disp('Number of iterations:');
    disp(iter);
    disp('Error of the last iteration:');
    disp(last_error);
    disp('--- SOR Iteration Method Outp. End ---');
end
```

chap-3/P87_Q7.m:

```
D = diag([10, 10, 10]);
L = [0, 0, 0; 4, 0, 0; 4, 8, 0];
U = L';
```

```
b = [13; 11; 25];
I = eye(size(D));
A = D + L + U;

% Jacobi

M = I - D \ A;
g = D \ b;

disp('--- Jacobi M & g ---');
disp(rats(M));
disp(rats(g));
eigenvalues = eig(M);
spectral_radius = max(abs(eigenvalues));
disp(spectral_radius);

% Gauss-Seidel
M =- (D + L) \ U;
g = (D + L) \ b;

disp('--- Gauss-Seidel M & g ---');
disp(rats(M));
disp(rats(g));
eigenvalues = eig(M);
spectral_radius = max(abs(eigenvalues));
disp(spectral_radius);

% SOR
omega = 1.35;
M =- (D + omega * L) \ ((1 - omega) * D - omega * U);
g = omega * (D + omega * L) \ b;

disp('--- SOR M & g ---');
disp(rats(M));
disp(rats(g));
eigenvalues = eig(M);
spectral_radius = max(abs(eigenvalues));
disp(spectral_radius);
```

其中，`P87_Q2_outputs.txt` 记录了 P87.2 中迭代若干步后的输出结果。例如，当误差阈值取 10^{-3} 时，Jacobi 迭代在 29 步后得到

$$x \approx (1.1997,\ 1.3989,\ 1.5996,\ 0.7993)^{\mathrm{T}},$$

Gauss-Seidel 迭代和 SOR 迭代则用更少步数达到相同精度。这也与第三章作业中的理论分析一致。